Skema RBAC — Storefront & Panel Admin OSR

Dokumen teknis · v1.2 · 30 Oktober 2026

> **Riwayat:** v1.0 — RBAC storefront & panel admin. v1.1 — kebijakan keranjang wajib akun (tamu tidak dapat menambah barang atau checkout). v1.2 — **paritas tampilan**: keranjang tetap terkunci untuk tamu, tetapi storefront dirender *identik* dengan sesi pelanggan; klik **Tambah** memunculkan pop-up masuk, bukan tombol yang disembunyikan.

Dokumen ini menjelaskan rancangan kendali akses berbasis peran (RBAC) yang kini terpasang pada replika **Octarine Sandbox Replica (OSR)**: siapa yang boleh masuk, apa yang boleh dilakukan, bagaimana identitas dibuktikan, dan bagaimana setiap peristiwa tercatat. Cakupannya meliputi basis data, API `osr-core`, halaman storefront, serta panel admin.

---

1. Ringkasan

Sebelum revisi ini, OSR tidak memiliki autentikasi sama sekali:

- **Storefront** hanya memakai cookie anonim (`osr_sid`) untuk menyimpan keranjang di memori. Tidak ada akun, tidak ada riwayat pesanan, checkout bisa dilakukan siapa pun yang mengisi nama penerima.
- **Panel admin** terbuka total — siapa pun yang membuka `:3001` langsung melihat metrik, event webhook, anomali, dan tombol *replay*.

Sesudah revisi ini, ada tiga peran, lima belas izin granular, sesi token opaque, penguncian akun otomatis, jejak audit append-only, dan pemisahan tegas antara "boleh melihat" dan "boleh menjalankan".

Aspek	Sebelum	Sesudah
Akun	tidak ada	tabel `osr_user`, registrasi mandiri untuk pelanggan
Peran	tidak ada	`customer` < `staff` < `admin`
Izin	tidak ada	15 izin granular, dipetakan lewat tabel
Sesi	cookie anonim untuk keranjang	token 256-bit, disimpan sebagai SHA-256
Panel admin	terbuka	gerbang login + pemeriksaan izin per halaman
Order	anonim	terikat `user_id`, pelanggan hanya melihat miliknya
Tampilan tamu	tombol tambah disembunyikan	**identik** dengan sesi pelanggan; klik *Tambah* memunculkan pop-up masuk
Jejak	tidak ada	`osr_auth_event` (login, gagal, kunci, tolak izin, ubah peran)

---

2. Prinsip desain

1. **Keputusan izin tunggal di core.** Storefront dan panel admin hanya mengatur tampilan dan navigasi (tombol, menu, pengalihan); keputusan akhir selalu di `osr-core`. Menembak API langsung tanpa izin tetap ditolak 403.
2. **Izin, bukan nama peran.** Kode memeriksa `admin.action.replay`, bukan `if role == 'admin'`. Menambah peran baru cukup menambah baris di tabel pemetaan, tanpa menyentuh kode.
3. **Pendaftaran mandiri hanya untuk pelanggan.** Peran `staff` dan `admin` tidak pernah bisa didaftarkan lewat form publik — hanya dibuat oleh admin yang sudah masuk.

4. **Kata sandi tidak pernah disimpan mentah.** Hash `scrypt` (N=16384, r=8, p=1) dengan salt acak 16 bita per akun.
5. **Token sesi tidak pernah disimpan mentah.** Basis data hanya menyimpan `sha256(token)`, sehingga bocornya tabel sesi tidak bisa dipakai membajak sesi.
6. **Semua penolakan dicatat.** 403 bukan jalan buntu senyap — setiap penolakan meninggalkan baris di `osr_auth_event` berisi izin yang diminta dan jalur yang diakses.

---

3. Peran

Peran	Rank	Label	Cakupan	Didaftarkan sendiri
`guest`	0	Pengunjung (tanpa akun)	Storefront publik saja: katalog + detail produk + deskripsi/BOM. **Tidak** boleh keranjang, checkout, atau status	— (bukan akun)
`customer`	10	Pelanggan	Storefront: katalog, keranjang, checkout, riwayat pesanan sendiri	ya
`staff`	50	Staf Operasional	Panel admin **baca saja**: dasbor, event, anomali, order, baseline	tidak
`admin`	100	Administrator	Seluruh izin staf + menjalankan aksi + mengelola akun	tidak

`guest` bukan baris di tabel `osr_role` — ia adalah keadaan "tanpa sesi" (cookie `osr_auth` tidak ada atau sudah dicabut). Ia dicantumkan di sini supaya perilakunya punya kontrak eksplisit, bukan tersirat dari ketiadaan kode.

Penting: **gerbang keranjang bergantung pada sesi, bukan pada peran.** `admin` yang membuka storefront tanpa sesi storefront diperlakukan sama seperti tamu — pop-up masuk muncul. Panel admin (`:3001`, cookie `osr_admin`) dan storefront (`:3000`, cookie `osr_auth`) memakai sesi terpisah, sehingga masuk ke panel tidak otomatis membuat keranjang bisa dipakai.

Kolom `rank` dipakai untuk membandingkan kewenangan bila kelak dibutuhkan (mis. larangan menurunkan peran admin lain oleh staf). Kolom `is_self_service` adalah penjaga eksplisit agar registrasi publik tidak pernah bisa menghasilkan peran internal.

---

4. Matriks izin

Izin (kode & label)	Scope	Jalur	guest	cust	staff	admin
`store.catalog.view` — lihat katalog & detail produk	store	4.1	✔	✔	—	✔
`store.cart.manage` — kelola keranjang	store	—	—	✔	—	✔
`store.order.create` — buat order (checkout)	store	4.2	—	✔	—	✔
`store.order.view_own` — lihat status order sendiri	store	4.2	—	✔	—	✔
`account.manage` — kelola profil sendiri	store	—	—	✔	—	✔
`admin.dashboard.view` — lihat dasbor metrik	admin	—	—	—	✔	✔
`admin.events.view` — lihat event webhook	admin	4.2	—	—	✔	✔

Izin (kode & label)	Scope	Jalur	guest	cust	staff	admin
`admin.anomalies.view` — lihat panel anomali	admin	4.2	—	—	✔	✔
`admin.orders.view` — lihat seluruh order	admin	4.2	—	—	✔	✔
`admin.baseline.view` — lihat gate baseline paper	admin	—	—	—	✔	✔
`admin.action.sync` — jalankan tarik katalog	admin	4.1	—	—	—	✔
`admin.action.reconcile` — rekonsiliasi status	admin	4.2	—	—	—	✔
`admin.action.replay` — replay event log	admin	4.2	—	—	—	✔
`admin.users.view` — lihat daftar akun	admin	—	—	—	—	✔
`admin.users.manage` — buat/ubah peran/nonaktifkan	admin	—	—	—	—	✔

Jumlah izin per peran: **guest 1**, **customer 5**, **staff 5**, **admin 15**. Perhatikan bahwa `admin.users.view` sengaja **tidak** diberikan ke `staff`: staf operasional perlu melihat angka, bukan data pribadi akun. Kolom **cust** = `customer`.

Baris `guest` hanya punya izin baca katalog. Itu sebabnya seluruh perilaku "belum masuk" di storefront tetap bisa dijelaskan dengan matriks ini: yang tidak ada di kolom guest tidak boleh terjadi, dan penegakannya ada di `osr-core`, bukan hanya di tampilan.

---

5. Model data

Enam tabel baru dan satu kolom tambahan. Semua didefinisikan di `db/002_auth.sql` dan bersifat idempoten (`IF NOT EXISTS` / `ON CONFLICT`), sehingga aman dijalankan berulang di atas basis data yang sudah hidup.

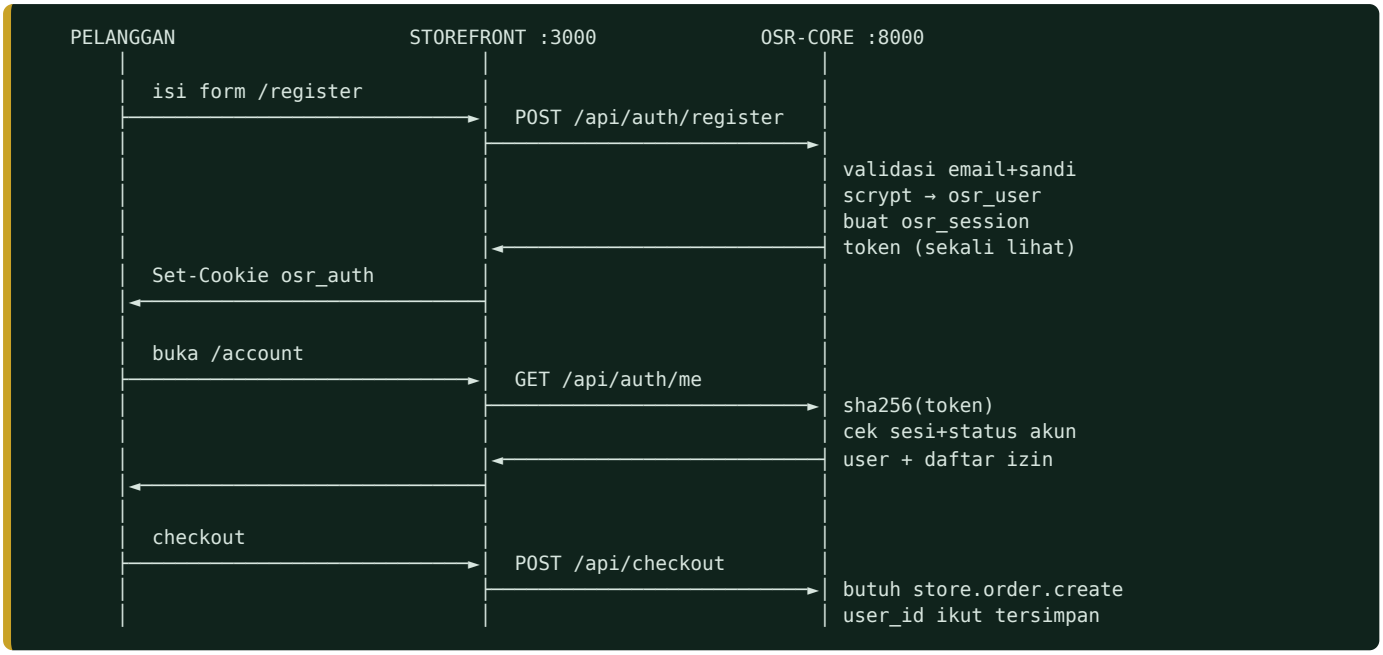
Tabel	Kunci	Isi	Catatan
`osr_role`	`role`	peran, label, peringkat, `is_self_service`	3 baris
`osr_permission`	`code`	kode izin, label, scope, jalur	15 baris
`osr_role_permission`	(`role`, `perm_code`)	pemetaan peran → izin	sumber kebenaran otorisasi
`osr_user`	`user_id`	email, nama, telepon, hash sandi, peran, status	unique index pada `lower(email)`
`osr_session`	`token_hash`	sesi aktif: ip, user agent, masa berlaku, pencabutan	token mentah tidak disimpan
`osr_auth_event`	`id`	jejak audit append-only	login, gagal, kunci, tolak, ubah peran

Kolom tambahan: `osr_order_attempt.user_id` (nullable, foreign key ke `osr_user`). Order tamu historis tetap sah karena kolomnya boleh kosong.

Status akun: `active` (boleh masuk), `suspended` (ditolak dengan pesan jelas), `pending` (belum diverifikasi — disiapkan untuk alur verifikasi email).

---

6. Alur autentikasi



Perbedaan penting antara kedua aplikasi:

- **Storefront** memakai cookie `osr_auth` berisi token. Pengunjung tanpa akun tetap melihat katalog **dan seluruh isi halaman produk** (nama, harga, stok, komposisi BOM, deskripsi) dengan tampilan yang sama seperti pelanggan. Yang dikunci hanya **keranjang**, **checkout**, **halaman akun**, dan **status pesanan**: klik *Tambah* oleh tamu memunculkan pop-up masuk, dan server tetap menolak tanpa sesi.
- **Panel admin** memakai cookie `osr_admin` berumur lebih pendek (12 jam). Bila akun yang login berperan `customer`, login **ditolak di halaman login** dengan pesan eksplisit — bukan dibiarkan masuk lalu ditolak belakangan. Sesi admin tidak berlaku di storefront: admin yang ingin memakai keranjang harus masuk lewat storefront juga.

---

7. Kontrak API

Metode	Jalur	Butuh	Keterangan
POST	`/api/auth/register`	—	Daftar pelanggan; peran dipaksa `customer`; balikan 201 + token
POST	`/api/auth/login`	—	Balikan 200 + token, atau 401 / 423 bila terkunci
POST	`/api/auth/logout`	sesi	Mencabut sesi aktif (`revoked_at`)
GET	`/api/auth/me`	sesi	Profil + daftar izin efektif
GET	`/api/auth/permissions`	sesi	Rincian izin beserta label & jalur
GET	`/api/my/orders`	`store.order.view_own`	Riwayat pesanan milik akun sendiri
GET	`/api/catalog` , `/api/catalog/{sku}`	—	Publik: katalog & detail produk boleh dilihat tamu (termasuk `components[]`)
POST	`/api/checkout`	`store.order.create`	Nama pemesan diambil dari akun, bukan form; tamu = 401
GET	`/api/orders`	`admin.orders.view`	Seluruh order

Metode	Jalur	Butuh	Keterangan
GET	`/api/orders/{ref}`	sesi	Admin/staf: semua; pelanggan: hanya miliknya (403 bila bukan)
GET	`/api/metrics`	`admin.dashboard.view`	Metrik dua jalur + DPMO
GET	`/api/events`	`admin.events.view`	Cermin event webhook
GET	`/api/anomalies`	`admin.anomalies.view`	Panel anomali
GET	`/api/paper-baseline`	`admin.baseline.view`	Gate reproduksi angka paper
POST	`/api/catalog/sync`	`admin.action.sync`	Jalur A
POST	`/api/reconcile`	`admin.action.reconcile`	Jalur B′
POST	`/api/replay`	`admin.action.replay`	Replay event log
GET	`/api/admin/users`	`admin.users.view`	Daftar akun + hitungan per peran
POST	`/api/admin/users`	`admin.users.manage`	Buat akun peran apa pun
PATCH	`/api/admin/users/{id}`	`admin.users.manage`	Ubah peran, status, nama, telepon, sandi
GET	`/api/admin/auth-events`	`admin.users.view`	Jejak autentikasi

Token dikirim lewat header `Authorization: Bearer <token>` (atau `X-OSR-Session`). Webhook masuk dari ERP (`POST /api/erp/webhook`) tidak memakai sesi pengguna karena bukan aktor manusia — kanal itu diautentikasi di lapisan proxy/ERP.

---

8. Perilaku antarmuka

Storefront

Halaman	Akses	Isi
`/`	publik	katalog + pencarian; markup **identik** untuk tamu dan pelanggan, termasuk tombol *Tambah* di setiap kartu
`/product/{sku}`	publik	detail lengkap: nama, harga, stok, deskripsi, dan komposisi BOM untuk bundle; tombol *Tambah ke keranjang* + kontrol qty tetap dirender untuk tamu
`/cart`	**sesi wajib**	tamu dialihkan ke `/login?next=/cart&reason=cart`; isi keranjang, hapus item, checkout
`/login`, `/register`	publik	form; setelah sukses langsung diarahkan ke `next`; menampilkan penjelasan bila `reason=cart`
`/account`	sesi	profil + riwayat pesanan + tautan lacak
`/status/{ref}`	sesi	riwayat status; pelanggan lain ditolak
`/logout`	sesi	cabut sesi, hapus cookie

Header berubah sesuai keadaan: tamu melihat *Masuk · Daftar*; pengguna yang sudah masuk melihat *Hai, {nama depan} · Keluar*, dan tautan **Admin** hanya muncul bila akun memiliki `admin.dashboard.view`. Angka keranjang `Keranjang (n)` hanya muncul bila ada sesi — tamu tidak punya isi keranjang untuk dihitung.

Kebijakan keranjang — terkunci, tetapi tampilan tidak berubah. Menjelajah katalog dan membaca deskripsi produk tetap publik sepenuhnya. Yang dikunci adalah **tindakan** menambah barang, bukan **tampilan** halaman:

- **Paritas markup.** Halaman katalog dan halaman produk dirender dari satu fungsi yang sama (`addToCartForm`) untuk tamu maupun pelanggan. Tidak ada tombol yang disembunyikan, tidak ada *notice* tambahan, tidak ada elemen pengganti. Yang berbeda hanya atribut gerbang `data-gate-form` / `data-gate-add` — tidak terlihat oleh pengguna dan tidak mengubah tata letak.
- **Klik *Tambah* oleh tamu → pop-up masuk**, bukan navigasi halaman. Pop-up memuat email + kata sandi, tautan *Daftar*, dan penjelasan singkat; bisa ditutup lewat tombol ×, klik latar, atau `Esc`.
- **Niat pengguna tidak hilang.** SKU dan qty yang tadi dipilih disimpan di sisi klien; setelah berhasil masuk, barang itu langsung dikirim ke `POST /cart/add` dan pengguna mendarat di `/cart` — bukan dikembalikan ke katalog untuk memilih ulang.
- **Enter di kolom qty juga tercegat**, bukan hanya klik tombol: form tamu menangkap event `submit` dan membuka pop-up yang sama.
- **Tanpa JavaScript tetap benar.** Bila JS mati, klik *Tambah* mengirim form seperti biasa; server menolak dengan `302` ke `/login?next=…&reason=cart`, lalu setelah masuk pengguna dikembalikan ke halaman asal. Jadi tidak ada jalan masuk lewat "matikan JS".
- **Penolakan dua lapis.** Storefront menangani UX (pop-up/redirect); `osr-core` tetap menolak kedua kalinya lewat izin `store.cart.manage` dan `store.order.create`. Menembak endpoint core langsung tanpa sesi tetap 401/403.
- **Gerbang mengikuti sesi, bukan peran.** `customer`, `staff`, maupun `admin` yang belum punya sesi storefront mendapat pop-up yang sama. Sebaliknya, siapa pun yang sudah punya sesi storefront (dan punya izin keranjang) bisa menambah barang.

Konsekuensi yang disengaja: tidak ada lagi order tamu baru, sehingga setiap order selalu punya `user_id` dan riwayat status bisa ditelusuri ke pemiliknya. Order tamu historis dari data produksi tetap sah karena kolom `user_id` boleh kosong.

Bukti paritas (terukur). `tools/check_guest_parity.py` mengambil HTML `/` dan `/product/{sku}` dari dua sesi berbeda, membuang bagian yang memang berbeda karena status (pop-up, `<script>`, atribut gerbang, sambutan header, angka keranjang), lalu membandingkan sisanya dengan `difflib.SequenceMatcher`. Hasil uji terakhir: **rasio kemiripan 1.0000** untuk katalog dan halaman produk — 15 dari 15 pemeriksaan lulus. Laporan tersimpan di `artifacts/parity_report.json`.

Bukti uji browser (end-to-end). Skenario dijalankan di peramban sungguhan, bukan hanya `curl`: tamu membuka `/`, menekan *Tambah* pada kartu pertama → pop-up "Masuk untuk melanjutkan" muncul **tanpa navigasi** (URL tetap `/`), berisi kolom email + kata sandi dan tombol ×. Setelah kredensial pelanggan dikirim, pengguna mendarat di `/cart` berisi SKU yang tadi diklik (`M2510-AquacDigi-100ml-A1`, qty 1), lengkap dengan tombol *Checkout* — tanpa memilih ulang barang.

Panel admin

- Tanpa sesi → dialihkan ke `/login?next=…`.
- Login dengan akun `customer` → ditolak di halaman login dengan pesan jelas.
- Menu navigasi **disaring per izin**: staf tidak melihat menu *Akun* maupun *Jejak Auth*.
- Membuka URL yang tidak diizinkan langsung (mis. staf membuka `/users`) → halaman "Tidak berwenang" yang menyebut izin yang dibutuhkan.
- Halaman `/users`: admin bisa membuat akun, mengubah peran, menonaktifkan akun. Tombol aksi **tidak muncul** untuk baris akun sendiri, dan API juga menolak (409) bila dipaksa — mencegah admin mengunci dirinya sendiri.
- Halaman `/auth-events`: jejak append-only seluruh peristiwa autentikasi.

---

9. Aturan keamanan

Kendali	Penerapan
Hash kata sandi	`scrypt` N=16384, r=8, p=1, salt 16 bita, dibandingkan `hmac.compare_digest`
Token sesi	32 bita acak (`secrets.token_urlsafe`), disimpan sebagai SHA-256
Umur sesi	storefront 7 hari (diperpanjang bergulir), admin 12 jam
Penguncian	5 kegagalan berturut → kunci 15 menit (`locked_until`), dihitung ulang saat berhasil
Anti-enumerasi akun	email tak dikenal tetap menjalankan hash dummy agar waktu respons serupa, lalu 401 generik
Rotasi sesi	ubah peran, ganti sandi, atau nonaktifkan akun → seluruh sesi pengguna itu dicabut
Spoofing identitas	nama pemesan checkout diambil dari akun, bukan dari isian form
Kebocoran lintas akun	`GET /api/orders/{ref}` memeriksa kepemilikan; pelanggan lain mendapat 403
Jejak audit	`osr_auth_event` mencatat register, login_ok, login_fail, lockout, logout, role_change, access_denied
Rahasia di repo	kata sandi akun awal dapat di-override lewat env; contoh di `.env.example` dikomentari

Nilai default di atas cocok untuk sandbox luring. Untuk penggunaan nyata, tiga hal wajib ditambahkan lebih dulu: verifikasi email, token CSRF pada form, dan pembatasan laju per alamat IP.

---

10. Pengujian

Tiga skrip uji dijalankan terhadap sistem hidup — bukan pembacaan kode.

Berkas	Cakupan	Hasil
`tools/smoke_auth.py`	47 pemeriksaan API: login benar/salah, registrasi, validasi, matriks izin 6 halaman × 2 peran, aksi tulis, checkout terikat akun, isolasi order, manajemen akun, logout, jejak audit	**47 lulus, 0 gagal**
`tools/smoke_ui.sh`	26 pemeriksaan halaman: form store, sesi, redirect, gerbang admin, penolakan akun pelanggan, penyaringan menu staf, buat akun, logout, gerbang keranjang tamu	**26 lulus, 0 gagal**
`tools/smoke_guest_cart.sh`	31 pemeriksaan kebijakan keranjang + paritas: tamu ditolak di `/cart/add` , `/cart` , `/cart/remove` , `/checkout` ; jumlah form/tombol tamu = pelanggan; deskripsi produk terbaca; admin tanpa sesi storefront juga tercegat; pelanggan berhasil menambah & checkout	**31 lulus, 0 gagal**
`tools/check_guest_parity.py`	15 pemeriksaan paritas HTML `/` dan `/product/{sku}` : normalisasi lalu bandingkan `difflib.SequenceMatcher` , plus cek isi wajib (nama, harga, stok, BOM)	**15 lulus, 0 gagal · rasio 1.0000 / 1.0000**

Perintah: `make auth-smoke` (RBAC), `make guest-smoke` (kebijakan keranjang + paritas), `make parity` (paritas saja), `make auth` (terapkan skema + akun awal), `make auth-reset` (bersihkan akun uji).

Contoh pemeriksaan yang dijalankan:

- `customer` → `GET /api/metrics` = **403**; `admin` → **200**
- `customer` → `POST /api/replay` = **403**; tanpa token = **401**

- checkout tanpa login = **401**; setelah login = **200** dan `user_id` terisi
- tamu → `POST /cart/add` = **302** ke `/login?next=…&reason=cart`; keranjang pelanggan terisi setelah masuk
- tamu → `GET /cart` = **302**; halaman login menampilkan penjelasan `reason=cart`
- katalog tamu merender **jumlah form dan tombol yang sama** dengan sesi pelanggan (48 = 48 pada seed saat uji)
- halaman produk tamu memuat `<h1>`, harga, `stok tersedia`, dan tabel *Komposisi* untuk bundle — sama seperti pelanggan
- selisih HTML setelah normalisasi = **0** (rasio 1.0000) untuk katalog dan halaman produk
- `admin` tanpa cookie storefront → pop-up muncul; setelah login storefront → pop-up hilang
- pelanggan A membuka order pelanggan B = **403**; admin = **200**
- registrasi dengan email duplikat = **409**; email tidak sah = **422**; sandi < 8 = **422**
- login pelanggan di panel admin = ditolak dengan pesan eksplisit
- staf membuka `/users` = "Tidak berwenang"; menu *Akun* tidak dirender
- `PATCH` peran akun sendiri = **409** (pagar anti self-lockout)
- audit memuat `login_ok`, `login_fail`, `register`, `logout`, `access_denied`, `role_change`

---

11. Operasional

```
make auth          # terapkan db/002_auth.sql + buat 3 akun awal
make auth-smoke    # 47 cek API + 26 cek halaman
make guest-smoke   # 31 cek kebijakan keranjang + paritas tampilan
make parity        # 15 cek paritas HTML tamu vs pelanggan
make auth-reset    # hapus akun uji, sisakan 3 akun bawaan
make reset         # db + auth + seed dari nol
```

Akun bawaan (kata sandi dapat di-override lewat env sebelum `make auth`):

Email	Peran	Kata sandi default
`admin@octarine.local`	admin	`Admin#2026`
`staff@octarine.local`	staff	`Staff#2026`
`pelanggan@octarine.local`	customer	`Pelanggan#2026`

Kredensial di atas hanya untuk sandbox luring; jangan dipakai ulang di lingkungan yang dapat diakses publik.

---

12. Batas yang diketahui

1. **Belum ada verifikasi email.** Kolom `email_verified` dan status `pending` sudah disiapkan, tetapi belum ada alur kirim tautan verifikasi.
2. **Belum ada lupa kata sandi.** Penggantian sandi saat ini hanya lewat admin (`PATCH /api/admin/users/{id}`).
3. **Belum ada token CSRF eksplisit.** Mitigasi saat ini bersandar pada `SameSite=Lax` dan pemeriksaan sesi; untuk produksi perlu token per form.
4. **Pembatasan laju masih per akun, belum per IP.** Penyerang yang mencoba banyak email berbeda belum tertahan.
5. **Belum ada 2FA.** Disediakan tempatnya di `osr_auth_event`, belum ada implementasi.

6. **Keranjang masih di memori proses.** Restart `osr-store` mengosongkan keranjang; riwayat pesanan tidak terpengaruh karena tersimpan di Postgres.
7. **Pop-up bergantung pada JavaScript.** Tanpa JS, tamu tidak melihat pop-up melainkan diarahkan ke halaman `/login` — perilaku tetap benar (server menolak), hanya jalur UX-nya berbeda. Karena itu `noscript` tidak lagi dipakai untuk mengubah markup tombol: tampilan harus tetap sama.
8. **Paritas diuji pada HTML server, bukan piksel.** Uji membandingkan markup yang dikirim server setelah normalisasi bagian dinamis. Beda visual yang murni CSS (mis. tinggi header karena nama pengguna lebih panjang) tidak tertangkap uji ini.

---

13. Berkas yang berubah

```
dmaic-training/octarine-sandbox/
├── db/
│   └── 002_auth.sql              BARU  — 6 tabel RBAC + kolom user_id
├── osr-core/
│   ├── auth.py                   BARU  — hash, sesi, dependency require()
│   └── app.py                    UBAH  — 8 endpoint auth + 9 endpoint dijaga
├── osr-store/
│   └── server.js                 UBAH  — /login /register /account /logout,
│                                         gerbang keranjang & checkout,
│                                         addToCartForm() paritas markup,
│                                         LOGIN_MODAL + GATE_JS (pop-up)
├── osr-admin/
│   └── server.js                 UBAH  — gerbang login, /users, /auth-events
├── tools/
│   ├── seed_users.py             BARU  — 3 akun awal idempoten
│   ├── smoke_auth.py             BARU  — 47 cek API
│   ├── smoke_ui.sh               BARU  — 26 cek halaman
│   ├── smoke_guest_cart.sh       UBAH  — 31 cek kebijakan keranjang + paritas
│   └── check_guest_parity.py     BARU  — 15 cek paritas HTML (difflib)
├── Makefile                      UBAH  — target auth, auth-smoke, guest-smoke,
│                                         parity, auth-reset
├── .env.example                  UBAH  — variabel sesi & sandi awal
└── artifacts/
    └── parity_report.json        BARU  — bukti rasio paritas 1.0000
```